

Week 11 • Talking to the backend well

Technical Interaction Design · ITU · Autumn 2026

Built 4 September 2026 · commit 4e11e8b

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-11.pdf>

CONTENTS

Efficient Communication With the Backend

- Being Aware of Costs
- Querying the Backend Efficiently
- Real-Time Data Updates

Updater functions for state

- Why would you use this
- The other way the setter surprises you: editing what is already in state

Running Code Server-Side

- Introduction: Client vs Server Code
- Cloud Functions in Parse
- Database Triggers

Efficient Communication With the Backend

Source: [Lectures/Technical/Backend/Efficient-Communication-with-the-Backend.md](#) — something unclear? [Open an issue](#), or send a pull request.

The separation between front-end and back-end is new: one used to do web development, and they did everything.

Today we talk about being a *full stack* developer if you can do both the backend and the front-end.

It is more satisfying to be full stack. And it's not much harder. This is because front-end is by definition event-driven, concurrent programming. Back-end is usually modeling and querying the DB.

This is why, it's worth for you, as naturally front-enders to have at least basic knowledge of backend. In this chapter we'll look at a few issues that are relevant for Parse Platform backends, but often will generalize to any kind of backend.

Being Aware of Costs

Back4App, the hosted service of Parse costs money. You are using a free tier, but you must realize that this is a cloud-based solution. For cloud-based solutions one has to pay.

Consequently, a free tier will have limitations. The limitations of the Back4App server ensure that you do not put too much pressure on the server.

Parse Server Pricing and Limitations

The image below shows the prices and limitations of the Back4App various plans.

Full Features List	Free	MVP \$15/month per app	Pay as you go \$80/month per app
	Start for Free	Buy Now	Buy Now
Requests/Month	25k	500k	5M
Data Storage	250MB	2GB	4GB
Data Transfer	1GB	250GB	1TB
File Storage	1GB	50GB	250GB
Apps per subscription	1	1	1
Daily backup	×	✓	✓

In particular for the free tier, you have limited:

- number of requests
- data transfer
- data storage

Impact on Design Decisions

Using a backend-as-a-service impacts how you design communication between your front-end and back-end. You have to be economical about resource usage because otherwise you:

- will spend more money than needed
- will have a slower application than necessary

Case Study: MathsCamp

Looking at the networking tab in the screenshot below, we can see many requests being made:

Tab groups

Maths Camp

Rangliste

Hjem

Kontakt

Log ud

Her er ranglisten for din klasse!

I dag

Denne uge

Al tid

RANG	MASKOT	NAVN	POINTS
1		Tammy2	725
2		PaulR	600
3		tedje	525
4		Sofie	525
5		testuser	300
6		mircealungu	175
-		Stense	150

Network

Preserve log

Disable cache No throttling

Filter

More filters

All Fetch/XHR Doc CSS JS Font Img Me

5000 ms 10000 ms

Name	St...	Ty...	Initiator	Size	Time
_User	204	pr...	Preflig...	0 B	105 ms
Points	204	pr...	Preflig...	0 B	105 ms
_User	200	xhr	Ranking.j...	93...	116 ms
Points	200	xhr	GetPoints	93...	113 ms
_User	204	pr...	Preflig...	0 B	104 ms
Points	204	pr...	Preflig...	0 B	103 ms
Points	200	xhr	GetPoints	93...	194 ms
_User	200	xhr	Ranking.j...	93...	194 ms
_User	204	pr...	Preflig...	0 B	100 ms
Points	204	pr...	Preflig...	0 B	100 ms
Points	200	xhr	GetPoints	1.1...	115 ms
_User	200	xhr	Ranking.j...	93...	115 ms
_User	204	pr...	Preflig...	0 B	132 ms
Points	204	pr...	Preflig...	0 B	133 ms
_User	200	xhr	Ranking.j...	93...	176 ms
Points	200	xhr	GetPoints	1.2...	175 ms
_User	204	pr...	Preflig...	0 B	99 ms
Points	204	pr...	Preflig...	0 B	99 ms
_User	200	xhr	Ranking.j...	93...	120 ms
Points	200	xhr	GetPoints	1.1...	119 ms
_User	204	pr...	Preflig...	0 B	108 ms
Points	204	pr...	Preflig...	0 B	108 ms
_User	200	xhr	Ranking.j...	93...	191 ms
Points	200	xhr	GetPoints	1.1...	190 ms
_User	200	xhr	Ranking.j...	93...	124 ms
_User	204	pr...	Preflig...	0 B	101 ms

348 / 379 requests 268 kB / 784 kB transferred

Discussion: Why might we need to improve this interaction? Is it to save money or is it to have a faster application?

Something unclear in this section? Open an issue.

Querying the Backend Efficiently

If you have *limited requests* it makes sense to get as much data from the DB in one go, as opposed to run multiple queries.

This is not a problem only for backends-as-a-service - this is a general problem. If you are not smart about querying your backend you will end up with a slow application.

The N+1 Select Problem

This is **one of the most popular ways** of using a DB sub-optimally.

You send a query, get n objects back, and then for every one of those objects, you send another query to get more information.

Why is this a problem?

- It's slower than it needs to be
- In the particular case of cloud-hosted Parse, you're running out of requests faster

How to solve it: Get all the data at once by *joining* the corresponding tables. This is, retrieving data from two tables at once.

Retrieving Data from Related Tables

By default, when you create a `Parse.Query` object you create it for a given table and you get back objects from that particular table. Pointers are retrieved only as the `objectId` of the actual object in the remote table. So normally, you'd have to send another query to bring also the object.

Solution: Use the `include` method of the `Parse.Query` object to specify that you want to include also the objects in the table pointed at by the pointers.

Example: Using `include` to fetch related objects

```
// Fetch all todos with their associated user information
let query = new Parse.Query('Todo');
query.equalTo('completed', false);
query.include('createdBy'); // createdBy is a pointer to a User object
let queryResult = await query.find();

// Now you can access the full user object
console.log(queryResult[0].get('createdBy').get('username'));
```

In this example:

- Without `include`: `createdBy` would only be a pointer (just the objectId)
- With `include('createdBy')`: The full `User` object is populated and you can access properties like username, email, etc.
- This avoids making separate queries for each user (N+1 problem)

Note: The corresponding SQL statement is `join`.

Field Selection: Retrieving Only Necessary Data

Just as sending too many queries is a problem, transferring from the server too much data that is not needed is also a problem. For example, if you need to display a simple list of todo titles, it would be a waste of *bandwidth* to transfer large description fields or attached files.

Solution: Use `exclude` and `select` methods to retrieve only the necessary fields from a table.

Example: Using `select` to get specific fields

```
// Fetch only title and completed status for a todo list view
let query = new Parse.Query('Todo');
query.select('title', 'completed');
let queryResult = await query.find();

console.log(queryResult[0].get('title'));           // Available
console.log(queryResult[0].get('completed'));       // Available
console.log(queryResult[0].get('description')) === undefined; // true - not fetched
```

Example: Using `exclude` to omit large fields

```
// Fetch all fields except the large description and attachments
let query = new Parse.Query('Todo');
query.exclude('description', 'attachments');
let queryResult = await query.find();

console.log(queryResult[0].get('title'));           // Available
console.log(queryResult[0].get('description')) === undefined; // true - excluded
```

When to use each:

- Use `select` when you need only a few specific fields (most common)
- Use `exclude` when you need most fields except a few large ones

Note: The corresponding SQL statement is `select title, completed from todos` instead of saying `select * from todos`.

Something unclear in this section? Open an issue.

Real-Time Data Updates

For most situations, we don't have to update our UIs when the data in the DB changes, because most of the time, a user is editing their own data and nobody else is changing it in the background.

When do we need real-time updates? One notable exception are situations when a screen is showing information that's created by other users that needs to appear in real time (e.g. messages in a chat application, or comments in a social media platform).

Polling: The Low-Tech Approach

How polling works:

- Set an [interval](#) (note that an interval is different than a [timeout](#))
- When the interval expires, request the data again from the server
- Redisplay if needed
 - make sure to specify the key for every of the items when rendering
 - update the list with the new data from the server (if the data has changed)

Important: Cleanup When Using Intervals in React

Make sure to use the special syntax of `useEffect` to **cleanup** the timer when the component is not visible anymore.

Wrong way:

```
useEffect(() => {  
  
  const interval = setInterval(() => {  
    // check for new messages  
  }, 1000);  
  
}, []);
```

Right way:


```
useEffect(() => {  
  // called with [] means on component load  
  
  const interval = setInterval(() => {  
    // check for new messages  
  }, 1000);  
  
  // on component unload  
  return () => {  
    clearInterval(interval);  
  };  
}, []);
```

Event-Driven Approaches: The Advanced Way

How event-driven updates work:

- Allow the server to send you events when something in your query has changed
- Update only when something has actually changed

Parse LiveQuery

Parse provides LiveQuery to enable real-time updates in your application. Instead of repeatedly asking the server “has anything changed?”, the server notifies you automatically when data matching your query changes.

Example: Real-time todo updates

```
// Step 1: Create a query for todos you want to monitor
let query = new Parse.Query('Todo');
query.equalTo('assignedTo', Parse.User.current());

// Step 2: Subscribe to the query
let subscription = await query.subscribe();

// Step 3: Handle different events

// When a new todo is created that matches the query
subscription.on('create', (todo) => {
  console.log('New todo created:', todo.get('title'));
  // Update your UI to show the new todo
});

// When an existing todo is updated
subscription.on('update', (todo) => {
  console.log('Todo updated:', todo.get('title'));
  // Update the specific todo in your UI
});

// When a todo is deleted
subscription.on('delete', (todo) => {
  console.log('Todo deleted:', todo.id);
  // Remove the todo from your UI
});

// When a todo enters the query results (e.g., assigned to you)
subscription.on('enter', (todo) => {
  console.log('Todo now matches query:', todo.get('title'));
  // Add the todo to your UI
});

// When a todo leaves the query results (e.g., reassigned to someone else)
subscription.on('leave', (todo) => {
  console.log('Todo no longer matches query:', todo.get('title'));
  // Remove the todo from your UI
});
```

Important: Cleanup in React

When using LiveQuery in React, make sure to unsubscribe when the component unmounts:

```
useEffect(() => {
  let subscription;

  const setupLiveQuery = async () => {
    const query = new Parse.Query('Todo');
    query.equalTo('assignedTo', Parse.User.current());
    subscription = await query.subscribe();

    subscription.on('create', (todo) => {
      // Handle new todo
    });

    subscription.on('update', (todo) => {
      // Handle update
    });
  };

  setupLiveQuery();

  // Cleanup: unsubscribe when component unmounts
  return () => {
    if (subscription) {
      subscription.unsubscribe();
    }
  };
}, []);
```

Benefits of LiveQuery over Polling:

- Real-time updates without constant querying
- Reduced server load (fewer requests)
- Lower latency (instant updates)
- More efficient bandwidth usage

How It Works: WebSockets (Optional Deep Dive)

Traditional HTTP requests work like sending letters:

- Client asks a question → waits for response → connection closes
- To get updates, must keep asking “anything new?” over and over
- Each request has overhead (connection setup, headers, etc.)

WebSocket connections work like a phone call:

- Client establishes a connection that stays open
- Server can send messages anytime without being asked
- Two-way communication channel remains active
- Much more efficient for real-time updates

LiveQuery uses WebSocket connections under the hood. This is why it can push updates to your client instantly without polling.

Comparison:

Using LiveQuery (what you actually use):

```
let subscription = await query.subscribe();
subscription.on('create', (todo) => {
  // Handle new todo
});
```

Using raw WebSockets (what LiveQuery does under the hood):

```
// You would need to manually:
// - Connect to the WebSocket server
// - Send properly formatted messages
// - Parse incoming JSON responses
// - Handle reconnection on disconnect
// - Manage subscriptions and unsubscriptions
// Thankfully, Parse LiveQuery handles all of this for you!
```

Parse implements this through the LiveQuery [protocol](#), but you don't need to understand the low-level protocol details to use it effectively.

Something unclear in this section? Open an issue.

Exam Questions

1. What is the N+1 select problem and why is it important?
2. How do you solve the N+1 problem in Parse?
3. What is the difference between polling and event-driven (LiveQuery) approaches for real-time updates?
4. What do `select` and `exclude` do in Parse queries?

```
query.select('title', 'completed');  
// vs  
query.exclude('description', 'attachments');
```

5. Explain this LiveQuery subscription:

```
subscription.on('enter', (todo) => {  
  console.log('Todo entered query:', todo.get('title'));  
});
```

6. What are WebSockets and why does LiveQuery use them?

Updater functions for state

Source: [Lectures/Technical/React/Updater-Functions-for-State.md](#) — something unclear? [Open an issue](#), or send a pull request.

You do not need this yet. `setTodos([ ...todos, item])` is correct wherever you read the state and write it back in the same breath — adding, deleting, toggling. The updater form earns its keep only when the value you captured has gone stale: two setter calls in one handler, or a callback that fires later — a response arriving, a timer, a live query pushing a change.

State can be updated with either **updater functions** like in the first button below or by calling the setter function with an actual value, the second button below. One increments with three the value of the state var. The other with one.

```
export default function Counter() {

  const [number, setNumber] = useState(0)

  return (
    <
      <button onClick={() => {

        // updater function that takes a callback
        // all the ones below are the same
        // the name of the variable does not matter!
        setNumber(x => x + 1);
        setNumber(n => n + 1);
        setNumber(number => number + 1);
      }}>+3</button>

      <button onClick={() => {

        // direct update of variable
        // although you call it three times, value is incremented with 1!
        setNumber(number + 1);
        setNumber(number + 1);
        setNumber(number + 1);
      }}>+1</button>
    </>
  )
}
```

Updater Function vs. Calling with Actual Value

- difference is between passing a callback and an actual value to the react state updater
- the updater will do this work at a later point, we don't control when that happens

Why would you use this

- [Queueing A Series of State Updates](#)
- When a lambda function risks capturing a stale state

Something unclear in this section? [Open an issue.](#)

The other way the setter surprises you: editing what is already in state

Since week 2 you have been building new arrays and new objects rather than editing the ones you had:

```
setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
```

Here is what that habit was protecting you from. This is the version people write when they are in a hurry:

```
const copy = [...todos]; // a new array...
copy.find((t) => t.id === id).done = true; // ...holding the SAME objects
setTodos(copy);
```

The array is genuinely new, so React sees a change and re-renders, and the screen is right. **This code works**, which is the whole problem with it: nothing tells you off, so the habit survives.

What it did was reach into a to-do React was already holding and edit it where it lay. That costs you as soon as anything else is holding the same object:

- **Anything that kept the previous list** — an undo stack, or a comparison of before and after — is now holding a list whose contents changed underneath it. The past was rewritten.
- **Anything that skips work by comparing objects** — a memoised child, an effect with the item in its dependency array — sees the same object it saw last time and concludes nothing changed. The screen goes stale, and nothing looks wrong in your code.

You met this shape in week 2, with [props](#): nothing physically stopped you pushing onto an array a parent had handed you, and it was wrong anyway. React is built on the assumption that what you put in state is never edited afterwards. So:

Treat everything already in state as read-only. To change it, build the new version and hand *that* to the setter — which is what `map` with a spread has been doing all along.

Something unclear in this section? [Open an issue.](#)

Exam Questions

1. What will happen when the button is clicked three times?

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount(count + 1);  
    setCount(count + 1);  
    setCount(count + 1);  
  }  
  
  return <button onClick={handleClick}>{count}</button>;  
}
```

2. What will be displayed after clicking the “+3” button?

```
function Counter() {  
  const [number, setNumber] = useState(0);  
  
  return (  
    <button onClick={() => {  
      setNumber(n => n + 1);  
      setNumber(n => n + 1);  
      setNumber(n => n + 1);  
    }}>+3: {number}</button>  
  );  
}
```

3. When would you use an updater function vs. passing a direct value to setState?

4. This code gives the setter a brand-new array, the screen updates, and it is still wrong. Why?

```
const copy = [...todos];  
copy.find((t) => t.id === id).done = true;  
setTodos(copy);
```

Running Code Server-Side

Source: [Lectures/Technical/Backend/Running-Code-Server-Side.md](#) — something unclear? [Open an issue](#), or send a pull request.

Introduction: Client vs Server Code

One of the nice things is that with Parse we did not have to implement a server.

All the code runs on the client side until now. `Parse.Query`, `Parse.Object` - they're APIs that run in the context of our Application.

However, for some kinds of code, we will need to run it on a server.

When to Use Server-Side Code

You need to write code that *runs on the server* for:

- **Statistics and aggregations** that might involve the data of all the users (e.g., average rating of all the movies in the database; one does not want send all that data to the client; it is good to do the calculations on the server)
- **Calling 3rd party APIs** (especially those that require a secret API key that can't be hardcoded in the Javascript)
 - e.g. your application offers Google Translate functionality
- **Validation and side effects** making a check before a user writes something to the database or updating some other metadata after a write is done

Discussion: Think of examples of computational tasks from your own project that would benefit from server-side execution.

Something unclear in this section? [Open an issue](#).

Cloud Functions in Parse

In Parse, cloud functions are:

- Defined on the server and run in the context of the server
- Called from the JS client by name with parameters

Example: Calculating Category Statistics Across All Users

Calling from the Client

Example of how to call a `getCategoryStats` cloud function from the client:

```
const params = {category: "Important & Urgent"};
const stats = await Parse.Cloud.run("getCategoryStats", params);
console.log(`Average time: ${stats.avgTime}, Total tasks: ${stats.count}`);
console.log(`Completed in last 24h: ${stats.completedCount}`);
```

Implementation on the Server

Such a function assumes that on the server the implementation of the `getCategoryStats` is present. A possible implementation could be:

```
Parse.Cloud.define("getCategoryStats", async (request) => {

  // Calculate the date 24 hours ago
  const now = new Date();
  const oneDayAgo = new Date(now.getTime() - 24 * 60 * 60 * 1000);

  const query = new Parse.Query("TodoItem");
  query.equalTo("category", request.params.category);
  query.greaterThanOrEqualTo("createdAt", oneDayAgo);
  // Note: NO userId filter - we want stats across ALL users

  const results = await query.find();

  let totalTime = 0;
  let completedCount = 0;

  for (let i = 0; i < results.length; ++i) {
    totalTime += results[i].get("totalTime") || 0;
    if (results[i].get("done")) {
      completedCount++;
    }
  }

  return {
    count: results.length,
    avgTime: results.length > 0 ? totalTime / results.length : 0,
    totalTime: totalTime,
    completedCount: completedCount
  };
});
```

Why run this on the server?

- You don't want to send all users' todo data to the client (privacy + bandwidth)
- The calculation happens on the server where the data lives
- The client only receives the final statistics

Optimization tip: For expensive statistics like this, you could cache the results in a separate table and refresh them periodically (e.g., once per day) instead of recalculating on every request. This reduces server load and saves money.

Key observations:

- We can still use the `Parse.Query` API on the server
- The params sent from the client are stored in the `request.params`
- The currently logged-in user is available as `request.user` (not used in this example)
- The function is defined as `async` so it's returning a promise

Deploying Cloud Functions

To declare cloud functions in Parse one has to go to the `Dashboard` > `Cloud Code` and add it to the `main.js` file under the `cloud` folder (see [example](#)). Once declared, you have to use the `Deploy` button at the bottom of the editor.

Something unclear in this section? [Open an issue](#).

Database Triggers

Parse allows you to automatically run functions when data is saved to the database. This is useful for validation, data transformation, and maintaining consistency.

beforeSave: Running Code Before Writes

You can automatically run a function on the server before a user makes a write to the database using the `beforeSave` method of `Parse.Cloud`.

Example: Validating Todo Names

```
Parse.Cloud.beforeSave("TodoItem", (request) => {

  const name = request.object.get("name");

  if (!name || name.trim().length === 0) {
    throw new Parse.Error(400, "Todo name cannot be empty");
  }

  if (name.length > 200) {
    // Truncate and add a ...
    request.object.set("name", name.substring(0, 197) + "...");
  }

  // Ensure totalTime is never negative
  const totalTime = request.object.get("totalTime");
  if (totalTime < 0) {
    request.object.set("totalTime", 0);
  }
});
```

Discussion Questions

- What does the above code do?
- How could a user abuse your system if you don't implement the above code?
- If you check for name length in JavaScript in the UI of your application... why would you still have to check right before saving a TodoItem also on the server?
- Why is it important to validate on the server even if you validate on the client?

afterSave: Running Code After Writes

You can define a function that is automatically called *after* a user makes a write to the database using the `afterSave` method.

Example: Tracking Completed Tasks

```
Parse.Cloud.afterSave("TodoItem", async (request) => {  
  
    // Only run this logic if the task was just marked as done  
    if (request.object.get("done") === true &&  
        request.original &&  
        request.original.get("done") === false) {  
  
        // Create a log entry for analytics  
        const CompletionLog = Parse.Object.extend("CompletionLog");  
        const log = new CompletionLog();  
  
        log.set("userId", request.object.get("userId"));  
        log.set("category", request.object.get("category"));  
        log.set("timeSpent", request.object.get("totalTime"));  
        log.set("completedAt", new Date());  
  
        try {  
            await log.save();  
            console.log("Completion logged successfully");  
        } catch (error) {  
            console.error("Error logging completion: " + error.message);  
        }  
    }  
});
```

Discussion Questions

- What does the above code do?
- Why do we check `request.original`?
- How could this data be useful for analytics or reporting?
- What other actions might you want to trigger when a task is completed?

Something unclear in this section? Open an issue.

Exam Questions

1. When should you run code on the server instead of the client?

2. Explain what this cloud function does:

```
Parse.Cloud.beforeSave("TodoItem", (request) => {  
  const name = request.object.get("name");  
  if (!name || name.trim().length === 0) {  
    throw new Parse.Error(400, "Todo name cannot be empty");  
  }  
});
```

3. What is the difference between `afterSave` and `beforeSave` triggers?